

Özet

Type	Pdf
Reviewed	☐

Web 1→ kullanıcı etkileşime girmez www nin ilk aşaması

web 2→içerikler kullanıcılar tarafından oluşturulur

web 3→anlamsal web veriler makineler tarafından anlaşılır hale gelir

web 4→simbiyotik ağ.insanlarve AI kullanan makineler arasında etkileşim artar bu şu an hedeftir

ORM→Object relational mapping veritabanı ve oop dilleriyle(java,c#) arası köprüdür

.NET Framework

Framework:yazılımcılar için iskelet görevi görür.Her şeyi sıfırdan yazmaya gerek kalmaz,tekrar eden işleri ayzılımcı yerine yapar

.NET Core

.net framework sadece windowsta çalışırken bu ise her işletim sisteminde çalışabilir

çapraz platform desteği,açık kaynak kod,performans ve mikroservis mimarisi uyumlu olduğu için daha avantajlıdır.

mikroservis:tüm uygulamayı küçük servislere bölerek çalıştırma işlemi.mesela e ticaret uygulamamız olsun mesela databaseden ürün aramamız gerekiyor bunun için normal database sorgusu yapmak yerine elastic search gibi özelleştirilmiş bir arama yaparak verimi arttırabiliriz.

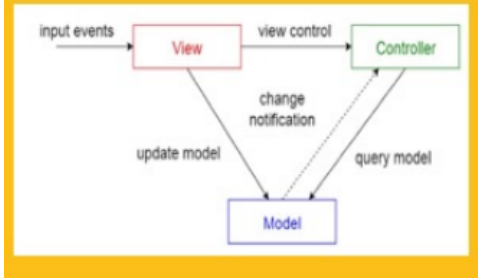
ASP.Net Core

web uygulama fremework. api ve mikroservis geliştirmek için kullanılır.Core dediği için .net core tabanlıdır ve cross-platform destekler.

Software Architectural Patterns

"Programın hangi parçalardan oluştuğunu ve bu parçaların birbirleriyle nasıl iletişim kurduğunu" anlatır.

MVC(Model view controller)

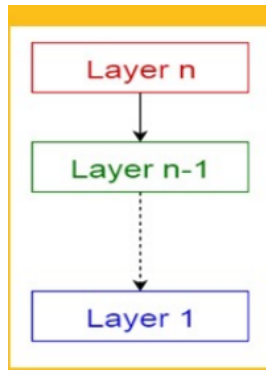


model verileri ve iş kurallarını tutar
mesela dbde kullanıcı bilgisi gibi.

View html sayfası gibi UI tutar

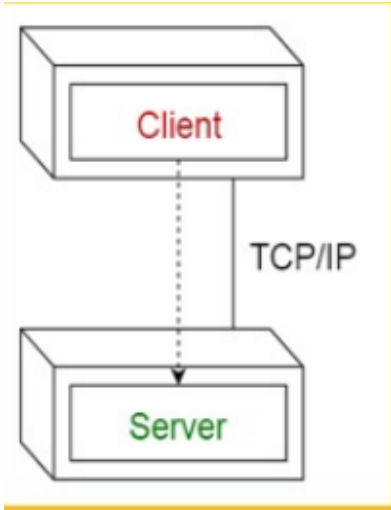
Controller kullanıcından gelen isteği alır
uygun modeli çağırır viewa gönderir
mesela kayıt ol butonu ne triggerlayacak
gibi

Layered Pattern



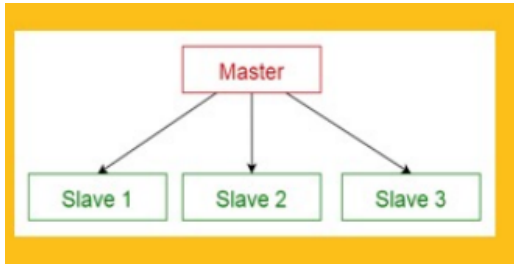
her katman farklı bir işlem gerçekleştirir
üst katman alt katmana istek gönderir ve
zincir yapısında çalışır.mvc daha çok
web layered ise hepsinde kullanılır
aslında mvc layeredın bir türüdür

Client server pattern



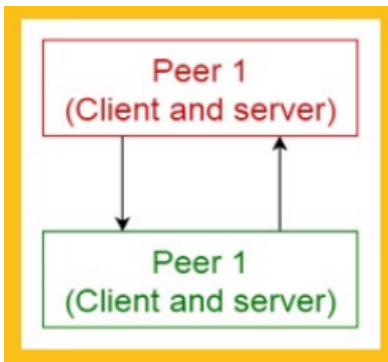
client istek gönderir serversa isteği alır işler cevap gönderir

Master Slave Pattern



master işleri planlar ve görev dağıtımı yapar slave ise görevleri yapar genelde büyük tekrarlı işleri hızlandırır mesela büyük veriyi bölüp paralel işlemek gibi

Peer to Peer pattern



hem pc hem client sunucu gibi davranır yani hem veri alabilir hem de veri paylaşabilir.

Nasıl Çalışır?

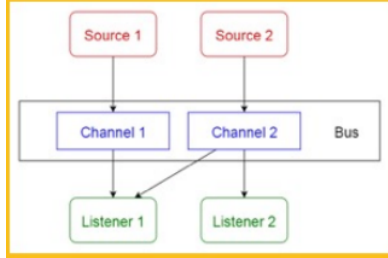
Diyelim ki bir dosya paylaşım ağı var:

- Senin bilgisayarında dosya "A" var,
- Başka birinin bilgisayarında dosya "B" var.

Sen "B" dosyasını indirirken aynı anda kendi "A" dosyanı başkasına gönderiyorsun.

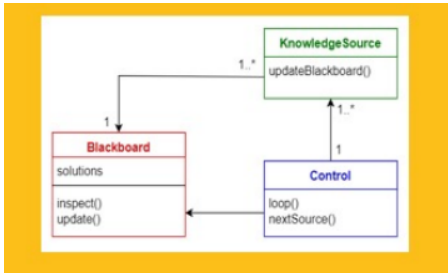
Böylece herkes birbirine **yardımcı** oluyor — merkezi sunucuya gerek kalmıyor.

Event Bus Pattern



uygulamadaki bileşenler birbirine doğrudan bağlı olmadan haberleşir. bir bileşen event yaratır diğeri dinler ve bus(veri yolu) üzerinden gerçekleşir Bir kullanıcı giriş yapınca (Source), sistem "LoginEvent" olayını yayınlar. Bu olayı dinleyen (Listener) bileşenler örneğin "**Bildirim Gönder**" veya "**Dashboard'u Aç**" olabilir.

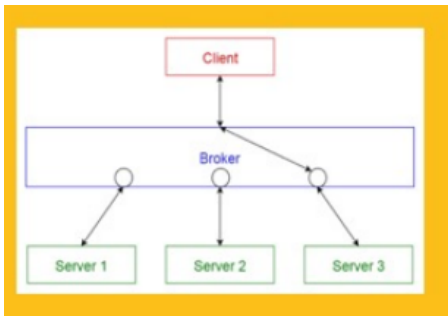
Blackboard Pattern



ortak veri alanı üzerinde birbirinden bağımsız modüllerin birlikte çalıştığı yapıdır Bir "tahta" düşün — herkes bu tahtayı görebiliyor, üzerine yazabiliyor.

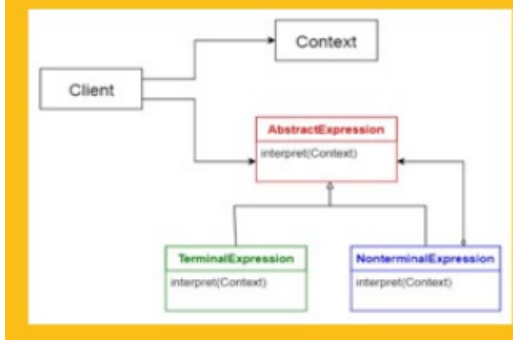
Ama hiçbiri doğrudan birbirini çağırıyor, sadece tahtayı kullanıyor her bileşen tahtaya bakar bir şey yapabilir miyim diye varsa tahtayı günceller controller hangi bileşenin sıradaki adımı yapacağını belirler

Broker pattern

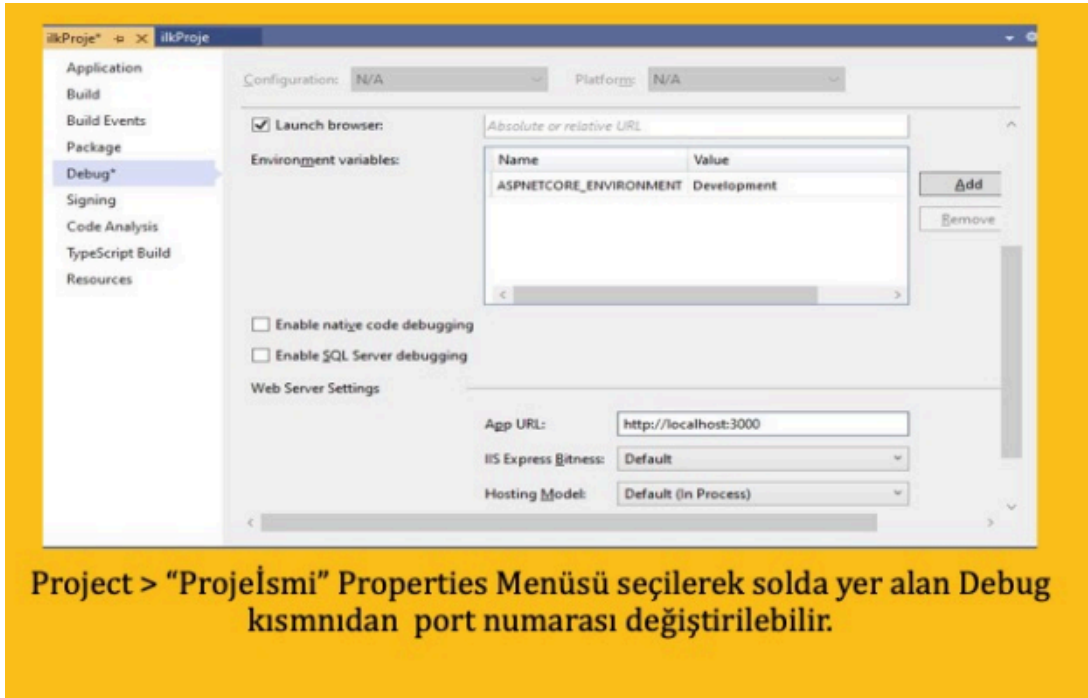


birbirinden uzak ya da bağımsız çalışan yazılımlar arası iletişimi organize eder mesela client servera direkt bağlanmaz broker araya girer mikroservis ya da distributed sistemlerde sıklıkla kullanılır

Interpreter Pattern



özel bir dilde yazılmış ifadeleri yorumlar(interpreter) mesela $2+3*4$ yazarsan doğru sırayla değerlnedir. yani hersembol ve ifadenin kuralları için sınıf oluşturu + - * if else vs vs



Project > "Projelismi" Properties Menüsü seçilerek solda yer alan Debug kısmından port numarası değiştirilebilir.

Model Bileşeni

Model, MVC'de uygulamaya ait verileri, uygulamanın durumunu ve uygulama tarafından gerçekleştirilmesi gereken tüm iş mantığını (business logic) veya işlemleri temsil eder. Proje yapısı ve büyüklüğüne göre tek katman veya birden fazla katmandan da oluşabilir.

View Bileşeni

View, içeriğin kullanıcı arabirimi aracılığıyla sunulmasından sorumludur. MVC'de projenin arayüzleri bu bölümde oluşturulur. .NET kodunu HTML içersine yerleştirmek için Razor View Engine (Razor görünüm motoru) kullanır. İçersinde minimum mantıksal işlemler yer almalıdır.

Projenin geliştirildiği yazılım dillerine göre dosya uzantıları da değişebilmektedir. Karmaşıklık gidermek adına View'ların yer aldığı klasörlerinin hiyerarsisi iyi ayarlanmalıdır. (HTML, Javascript, CSS ve resim dosyalarını aynı klasör içinde barındırmamak vb)

Controller Bileşeni

Controller, kullanıcı etkileşimini yöneten, modelle çalışan ve sonuçta oluşturulacak görünümü seçen bileşenlerdir. Bu bileşen kullanıcıdan gelen istekleri(request) işler View ile Model arasında bağlantı kurarak kullanıcıya hangi View'ın döneceği (response) belirtilir.

Kısaca MVC modelinde controller ilk giriş noktasıdır ve hangi model türleriyle çalışacağını ve hangi görünümün oluşturulacağını seçmekten sorumludur.

Routing

web uygulamasında url isteğini doğru controller cve actiona yönlendiren mekanizmadır.Kullanıcı tarayıcıya bir adres (URL) yazdığında, sistem o isteği nereye göndereceğini **routing** sayesinde bilir.

```
routes.MapRoute(  
    name: "Default",  
    template: "{controller=Home}/{action=Index}/{id?}"  
);
```

controller=Home: Eğer kullanıcı adres çubuğuna hiçbir şey yazmazsa, varsayılan controller "Home" olur.

action=Index: Controller içindeki "Index()" metodu çalıştırılır.

{id?}: İsteğe bağlı bir parametredir (örneğin "/Home/Details/5").

Yani /Home/Index URL'si → HomeController içindeki Index() metoduna gider.

Action Method

Controllerdaki public methodlardır.Public olmalıdırlar.static ve aşırı yüklenmiş methodlar action olamazlar.(static methodlar nesne oluşturulmadan sınıfın kendi içerisinde çağırılır ama controller nesne oluşturarak istekleri yerine getirir,controller url ismiyle çağırma yapar ama overloading yapılırsa mvc hangi url'i çağıracağını bilemez)bunu yaparken url'ye gidip kodu çalıştırıp sonucu elde eder.

```
public class HomeController : Controller  
{  
    // static olduğu için bu bir Action  
    Method olamaz ❌  
    public static string Index()  
    {  
        return "Merhaba Selva!";  
    }  
}  
doğru hali:
```

```
public class OgrenciController : Controller  
{  
    // 1. versiyon  
    public string Bilgi()  
    {  
        return "Öğrenci bilgisi yok";  
    }  
  
    // 2. versiyon (overloaded)  
    public string Bilgi(int id)
```

```
public string Index()
{
    return "Merhaba Selva!";
}
{
    return "Öğrenci ID: " + id;
}
}
```

hangi bilgi seçemez?

Örnek Bir Model Sınıfı

```
public class Oğrenci
{
    public string OğrAd { get; set; }
    public string OğrSoyad { get; set; }
    public string OğrNo { get; set; }
    public int OğrYas { get; set; }
}
```

Model, uygulamadaki **verileri ve bu verilerle ilgili kuralları** temsil eden sınıftır.

get → değeri okumaya (erişmeye) izin verir,

set → değeri değiştirmeye (yeni değer atamaya) izin verir.
sadece get olursa okunabilir ama değiştirilemez sadece set ise tam tersi

Layout

Aynı kalan header, left menu, footer vb. gbii şeyler

shared klasöründe → layout

view starta @{layout="layout"} denilerek kdo çalışmaya başlar

Razor View Engine

bir web uygulamasında kullanıcıya gösterilecek HTML çıktısını üretmekten sorumlu yapıdır çünkü kodlar html haric c cpp vb içerebilir

Html-C# için @ kullanır

Tarih : @DateTime.Now.ToString() şeklinde bir kod → @DateTime.Now.ToString() kodu Razor tarafından C# kodu olarak yorumlanıp ekrana basılacaktır.

(Tarih : 14.08.2021 16:33:28 gibi günün tarih ve saatini veren bir çıktı)

Tarih : DateTime.Now.ToString() şeklinde yazılmış olsaydı Herhangi bir yorumlama olmadan çıktı aynı şekilde ekrana basılacaktır.

(Çıktı : Tarih : DateTime.Now.ToString())

Razor - Koşullu İfade Örneği if / if-else

```
@{var fiyat = 100;}
<html>
<body>
    @if (fiyat > 50)
    {
        <p>Fiyat 50 lirada fazla.</p>
    }
</body>
</html>
```

```
@{
    var sayi1 = 100;
    var sayi2 = 200;
}
<div>
    @if (sayi1 > sayi2)
    {
        <p>Sayi1 Büyük</p>
    }
    else
    {
        <p>Sayi2 Büyük</p>
    }
</div>
```

```
@{var fiyat = 40;}
<div>
    @if (fiyat >= 100)
    {
        <h2>Fiyat Yüksek.</h2>
    }
    else if (fiyat > 30 && fiyat < 50)
    {
        <h2>Fiyat Normal</h2>
    }
    else
    {
        <h2>Fiyat düşük.</h2>
    }
</div>
```

```
@{
    string[] Ogrenciler = { "Semra", "Hasan", "Tuncay", "Mehmet" };
}
<ul>
    @for (int i = 0; i < Ogrenciler.Length; i++)
    {
        <li>@Ogrenciler[i]</li>
    }
</ul>
```

Kodun Ekran Görüntüsü	Kodun Html çıktısı
- Semra - Hasan - Tuncay - Mehmet	<pre><p> </p> <div> @if (sayi1 > sayi2) { <p>Sayi1 Büyük</p> } else { <p>Sayi2 Büyük</p> } </div></pre>

Razor- while döngüsü

```
@{
    string[] ogrenciler = { "Semra", "Hasan", "Tuncay", "Mehmet" };
    int i = 0;
}
<p>
<ul>
    @while (i < ogrenciler.Length )
    {
        @:
        <li>@ogrenciler[i]</li>
        i++;
    }
</ul>
</p>
```

Kodun Ekran Görüntüsü	Kodun Html çıktısı
- Semra - Hasan - Tuncay - Mehmet	<pre><p> </p> <div> @if (sayi1 > sayi2) { <p>Sayi1 Büyük</p> } else { <p>Sayi2 Büyük</p> } </div></pre>

Web Form

kullanıcıdan ver i alıp veriyi web sunucusuna gönderir

<input type="hidden">kullanıcılar göremez veya değiştiremez.mesela kaçınıcı post olduğunu öğrenemk isityrouz sadece developerın ihtiyacı var on ututar gibi

select→açılır liste textarea→etxtarea

Post-Get Methods

get→ veriler url sonuna eklenir ?başlama &anahtar ayırma

post→urlde gözükmeyen istek gövdesinde gönderilir

action→formda girilen bilgi nereye gönderilecek-yazılmazsa aynı sayfaya veri gönderir

method→hangi yöntemle gönderilecek

Model Binding Nedir?

Model Binding, formdan veya URL'den gelen verilerin **otomatik olarak C# değişkenlerine veya nesnelere aktarılmasıdır.**

ani artık:

csharp

Copy code

```
string ad = HttpContext.Request.Form["ad"];
```

gibi tek tek veri almak yok 💡

ASP.NET bunu otomatik yapıyor 🍌

◆ Örnek: URL Parametreleriyle Model Binding

csharp

Copy code

```
public string DegerAl(int? id, string ogrAd)
{
    return "id = " + id.Value + " ve Ad = " + ogrAd;
}
```

Eğer tarayıcıda şu URL çağılırsa:

arduino

Copy code

```
http://localhost:3000/Home/DegerAl/100?ogrAd=Hasan
```

ASP.NET şunu yapar:

- `id` parametresine **100** değerini verir
- `ogrAd` parametresine **Hasan** değerini verir

Durum Yönetimi(state management)

durumsuz ve bilgili vers. vardır

http statelessdir her komut önceki yürütülen komut hakkıdna bilgisi yoktur

istemci taraflı-sunucu taraflı ve ikisinin kullunıldığı seçenekler vardır

İstemci taraflı durum yönetimi teknikleri

Query String

url sonuna eklenir web sayfasından diğerine veri göndermek için kullanılır

Hidden Fields (Gizli Alanlar)

HTML'de `<input type="hidden">` etiketiyle tanımlanan, kullanıcıya görünmeyen ama form ile birlikte **veri gönderen** alanlardır.

```
<input type="hidden" name="userId" value="123">
```

Amaç: Web sayfaları arasında geçici veri saklamak ve sonraki istekte geri göndermek için kullanılır.

tutum kullanıcı idsi vb şeyleri saklar

🍪 HTTP Cookies (Çerezler)

web sunucusunun kullanıcının **tarayıcısına gönderdiği küçük veri**

parçalarıdır. Tarayıcı bu verileri metin dosyası olarak bilgisayarda saklar. Bir sonraki istekte bu bilgiler **sunucuya geri gönderilir**, böylece sunucu kullanıcının kim olduğunu hatırlayabilir. `Microsoft.AspNetCore.Http` kütüphanesi ile yapılır.

Sunucu Tarafı Durum Yönetimi

Session state

session (oturum), kullanıcının bir web sitesine giriş yaptığı andan siteyi terk ettiği ana kadar geçen süredir. Bu süre boyunca kullanıcıya özel veriler **sunucu tarafında saklanır**.

`Microsoft.AspNetCore.Session`

View a veri aktarımı

ASP.NET MVC modelinde, kullanıcıdan gelen HTTP isteğini

Controller karşılar. Controller, isteğe uygun işlemleri yaptıktan sonra sonucu

View'a gönderir. Bu nedenle verilerin **Controller → View**

yönünde aktarımı gerekir.

ViewBag

ViewBag, Controller'dan View'e veri aktarmanın en basit yollarından biridir.

Özellikleri:

- **Dinamik** bir nesnedir → yani önceden veri tipi belirlemeden kullanılabilir.

- ControllerBase sınıfından türetilmiştir.
- Controller çalıştığında bellekte oluşur, View render edildikten sonra silinir.

Controller içinde:

```
csharp
public IActionResult Index()
{
    ViewBag.Message = "Hoşgeldin Selva!";
    return View();
}
```

View içinde (.cshtml):

```
html
<h2>@ViewBag.Message</h2>
```

Sonuç:

Sayfa çalıştığında ekranda "Hoşgeldin Selva!" görüntülenir.

küçük geçici verilerde kullanılır mesaj uyarı vs

Viewdata

Veriler **Dictionary (sözlük)** türünde saklanır, yani **anahtar-değer (key-value)** çiftleri şeklindedir.

Özellikleri:

- `ViewData["key"] = value;` şeklinde veri atanır.
- Sadece **Controller** → **View** yönünde çalışır.
- Veriler **string anahtarlarla** tutulur.
- **Sadece mevcut istek sırasında** geçerlidir (sayfa yenilendiğinde kaybolur).
- `ControllerBase` sınıfından türetilmiştir.

Kullanım örneği:

csharp

Copy code

```
public IActionResult Index()
{
    ViewData["isim"] = "Selva";
    return View();
}
```

View içinde:

html

Copy code

```
<h3>@ViewData["isim"]</h3>
```

● Sonuç: Ekranda "Selva" yazısı görünür.

Tempdata

`TempData`, ViewData gibi Controller'dan View'e veri aktarır ama farkı, **bir sonraki istek için de geçerli** olmasıdır.

Özellikleri:

- `TempData` da **Dictionary** türündedir.
- `ControllerBase` sınıfının bir özelliğidir.
- ViewBag ve ViewData bir istekten sonra silinirken, TempData **sonraki sayfada da kullanılabilir**.
- Genellikle **sayfalar arası veri taşımak** veya **redirect** işlemlerinde kullanılır.

Kullanım örneği:

csharp

 Copy code

```
public IActionResult Index()
{
    TempData["not"] = 100;
    return RedirectToAction("Result");
}
public IActionResult Result()
{
    var not = TempData["not"];
    return View();
}
```

● Sonuç: "Result" sayfasına geçildiğinde `not` değişkeni hâlâ kullanılabilir olur.

Kısa Özet

Özellik	ViewBag	ViewData	TempData
Tür	Dinamik	Dictionary	Dictionary
Erişim süresi	Sadece mevcut istek	Sadece mevcut istek	Sonraki isteğe kadar
Kullanım yönü	Controller → View	Controller → View	Controller ↔ Controller, Controller → View
Kullanım amacı	Küçük geçici veriler	Küçük geçici veriler	Sayfalar arası veri taşımak

ViewBag, ViewData ve TempData

- ViewData ve TempData değerlere erişmek için dictionary sözdizimini (key-value) kullanırken, ViewBag property sözdizimini (dot notation) kullanır.
- ViewData ve TempData ile kullanılan değer, bir değişene aktarılacak istendiğinde tip dönüşümü yapılması gereklidir. ViewBag'de tip dönüşümüne gerek yoktur.

```
@{  
    string msj = (string)ViewData["Ad"];  
    string mj2 = ViewBag.OgrAd;  
    int a = (int)TempData["No"];  
}
```

- ViewData, ViewDataDictionary'den türetilmiştir, bu nedenle ContainerKey, Add, Remove ve Clear gibi yararlı olabilecek özelliklere sahiptir.

ViewBag, ViewData ve TempData

Özet fark:

- **ViewBag** ve **ViewData**, sadece **tek bir istek** süresince geçerlidir.
- **TempData**, sonraki istekte de (örneğin `RedirectToAction` sonrasında) kullanılabilir.

Yaşam döngüsü farkı:

- `ViewBag` / `ViewData` → sayfa yüklendiğinde oluşur, işlem bitince silinir.
- `TempData` → bir sonraki HTTP isteğine kadar yaşar.

Model Verilerinin Aktarılması (ViewModel)

MVC modelinde, Controller'dan View'e sadece basit değişkenler değil, **model sınıflarına ait veriler** de aktarılabilir.

Controller'da:

csharp

Copy code

```
public ActionResult Kaydet()
{
    Ogrenci ogr = new Ogrenci();
    ogr.Ad = "Selva";
    ogr.Soyad = "Artunç";
    ogr.Numara = 123;
    return View(ogr);
}
```

View'de (Kaydet.cshtml):

html

Copy code

```
@model Ogrenci
<p>@Model.Ad</p>
<p>@Model.Soyad</p>
<p>@Model.Numara</p>
```

Özetle:

Tag Helper

- **HTML etiketleriyle aynı yapıya sahiptir**, yani klasik HTML yazımını bozmadan C# özellikleri eklenir.
- **IntelliSense desteği** (otomatik tamamlama) sağlar.
- **HTML Helper'lara** göre daha modern ve okunabilir bir sözdizimi sunar.
- **Kod tekrarını azaltır**, HTML içinde uzun C# ifadeleri yazmaya gerek kalmaz.

HTML Helper (eski yöntem):

csharp

Copy code

```
@Html.TextBoxFor(m => m.OgrenciAdi, new { @class = "form-control" })
```

Tag Helper (yeni yöntem):

html

Copy code

```
<input asp-for="OgrenciAdi" class="form-control" />
```

● Sonuç:

Yukarıdaki Tag Helper, HTML'e çok benzer ama arka planda C#'ın gücünü kullanır.

Anchor Tag Helper

🔗 Anchor Tag Helper (Bağlantı Etiketi Yardımcısı)

- `<a>` etiketini **Controller** ve **Action** adlarını kullanarak yönlendirme yapar.
- **asp-controller**: Gidilecek controller'ı belirtir.
- **asp-action**: Çağrılacak action metodunu belirtir.
- **asp-route**: URL'ye parametre eklemek için kullanılır.

■ Örnek:

html

Copy code

```
<a asp-controller="Ogrenci" asp-action="Kaydet">Tıkla</a>
```

➡ `/Ogrenci/Kaydet` adresine yönlendirir.

Form Tag Helper

Form Tag Helper (Form Etiket Yardımcısı)

- `<form>` etiketine `asp-controller`, `asp-action` gibi özellikler ekleyerek MVC eylemlerine veri gönderir.
- Otomatik olarak CSRF güvenlik token'ı ekler → dış saldırılara karşı koruma sağlar.

■ Örnek:

html

Copy code

```
<form asp-action="Kaydet" asp-controller="Ogrenci" method="post">
    ...
</form>
```

➡ HTML'de otomatik olarak `/Ogrenci/Kaydet` adresine giden güvenli bir form oluşturur.

asp-page
Razor Pages kullanılıyorsa hedef sayfayı belirtir.

Copy code

```
<form asp-page="/Kayit">
```

asp-area
Uygulamada alan (area) yapısı varsa o bölgeyi belirtir.

Copy code

```
<form asp-area="Sirket" asp-controller="Home" asp-action="Delete">
```

➡ `/Sirket/Home/Delete` adresine yönlendirir.

asp-fragment
URL'ye # ile başlayan bir sayfa parçası (anchor) ekler.

Copy code

```
<form asp-action="Delete" asp-fragment="#10">
```

➡ `/Home/Delete#10` oluşturur.

asp-page
Razor Pages kullanılıyorsa hedef sayfayı belirtir.

Copy code

```
<form asp-page="/Kayit">
```

asp-host
URL'de özel bir host (sunucu adresi) tanımlar.

Copy code

```
<form asp-controller="Home" asp-action="Deneme" asp-host="http://test.com">
```

➡ `http://test.com` adresine yönlendirir.

asp-protocol
URL'de kullanılacak protokolü (http/https) belirtir.

Copy code

```
<form asp-protocol="https" asp-controller="Home" asp-action="Delete">
```

area admin user gibi kullanılır erişebileceği alanları ayırmayı kolaylaştırır aslında

Input taghelper

HTML formlarında kullanıcıdan veri almak için kullanılan `<input>` elemanlarını, **model özelliklerine (property)** bağlayarak otomatik oluşturur. Bu da kod tekrarını azaltır ve form yapısını **model odaklı** hale getirir.

Kullanım Örneği

Model sınıfı:

csharp

 Copy code

```
public class Ogrenci
{
    public string OgrAd { get; set; }
    public string OgrSoyad { get; set; }
    public int OgrNo { get; set; }
}
```

View'de:

html

 Copy code

```
<input asp-for="OgrAd" />
```

➔ Razor bunu otomatik olarak şu HTML'e dönüştürür:

html

 Copy code

```
<input type="text" id="OgrAd" name="OgrAd" value="" />
```

.NET türlerine karşılık oluşturulan HTML input özellikleri

.NET type	Input Type
Bool	type="checkbox"
String	type="text"
DateTime	type="datetime-local"
Byte	type="number"
Int	type="number"
Single, Double	type="number"

Label Tag Helper

Oluşturduğumuz Öğrenci sınıfına ait bir label tag helperi aşağıdaki gibi kullanılabilir.

```
<label asp-for="OgrSoyad" class="control-label"></label>  
<input asp-for="OgrSoyad" class="form-control" />
```

Yukarıdaki kod çalıştırıldığında input etiketinden önce bu input alanına ait otomatik olarak oluşturulan açıklama aşağıda gösterilmiştir.

OgrAd

```
[Display (Name ="Öğrenci Adı")]  
public string OgrAd { get; set; }
```

Öğrenci Adı

Display özelliği ile bir etikete için Label tag helperi ile bağlanan görüntülenen metin değerlerini istediğimiz şekilde oluşturmak için kullanılır.

textarea Tag helper

Kullanıcıdan **birden fazla satır metin** almak için kullanılır.

Select Tag Helper

`<select>` etiketi, kullanıcıya **açılır bir liste (dropdown)** sunmak için kullanılır.

aspnetcore.mvc.rendering kütüp kullanılmalı ISelectItem için

Scaffolding(Hazır razor şablon)

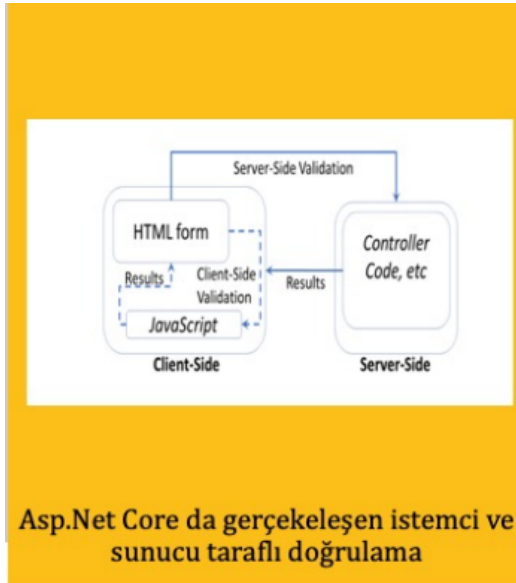
İde direkt crud yapar.

- **Empty:** HTML <body> kısmı boş bir Razor Sayfası oluşturur.
- **Create:** Belirtilen model sınıfını oluşturmak için form öğelerini içeren bir Razor Sayfası oluşturur.
- **Edit:** Belirtilen model sınıfını düzenlemek form elemanlarını barındıran razor sayfası oluşturur
- **Delete:** Seçili varlığın silinmesi için bu varlığa ait hidden bir alan ile bu varlığa ait benzersiz bir değerin tutulduğu bir form üretir.
- **Details:** Seçili varlığın ayrıntılarını görüntüleyen bir razor sayfa üretir.
- **List:** Seçili varlığın tüm örneklerinin ayrıntılarını görüntüleyen bir sayfa üretir

FINALLLLLL

Model Validation

Genek yaklaşım tüm kullanıcı bilgileri hatalı olması.



Verinin belirlenen kısıtlamalar dahilinde olup olmadığı → web sunucu tarafında

Tarayıcıda yapılan doğrulama → istemci doğrulaması

sunucuda yapılan doğrulama server side

Model Doğrulama

Uygulamadan alınan verilerin modele bağlamaya uygun olmasını sağlama

Model bağlama HTTP isteklerinden gelen verileri action methoda ait parametrelerle otomatik eşler doğrulamazsa istemci ve sunucuda doğrulamasını otomatik yapar

Model verilerine uygun olarak tag helper(view) istemcide

Controller içinde action controlleriyse sunucuda doğrulanır

System.ComponentModel.DataAnnotations

Doğrulama için Sık Kullanılan Öznitelikler

- **[Compare]** Bu öznitelik, model sınıfındaki iki özelliğin birbiri ile eşleşip eşleşmediğini tanımlar. (örneğin parola oluştururken parolanın iki kere girilmesi ve bunların eşleşip eşleşmediğinin kontrolü)

```
public string Sifre { get; set; }
[Compare("Sifre", ErrorMessage = "Şifreler eşleşmiyor")]
public string SifreDogrulama { get; set; }
```

- **[EmailAddress]** Özelliğin e-posta adresi biçimine sahip olup olmadığını doğrular.

```
[EmailAddress(ErrorMessage = "Geçerli bir mail adres giriniz")]
public string Email { get; set; }
```

- **[Phone]** Özelliğin geçerli bir telefon numarası biçimine sahip olup olmadığını doğrular. Eğer "-" vb gibi karakterle ayrılmış olarak özelleştirilmiş bir şekilde gösterilmek istenirse uygun bir regular expression ifadesi tanımlanmalıdır.

```
[Phone(ErrorMessage = "Lütfen Geçerli bir telefon giriniz")]
public string Telefon { get; set; }
```

- **[Range(min,max)]** Özellik değerinin belirtilen aralık değerlerinde olup olmadığını doğrular.

```
[Range(1, 120, ErrorMessage = "Yaş aralığı 1-120 olmalıdır.")]
public int Yas { get; set; }
```

- **[RegularExpression]** Özelliğin verilen Regular Expression ifadesine uyup uymadığını doğrular.

```
[RegularExpression("^?(0+)$\\+?\\d{1,3}[ ]?(?0+)$\\+?\\d{10,15}$",
ErrorMessage = "Telefon numarasını uygun formatta giriniz.")]
public string Telefon { get; set; }
```

- **[Required]** Özelliğin boş veya NULL hariç bir değere sahip olup olmadığını doğrular.

```
[Required(ErrorMessage = "Kullanıcı Adı Alanı Zorunludur")]
public string KullaniciAdi { get; set; }
```

Doğrulama Öznitelikleri Örnek Kullanım

```
public class Kullanici
{
    [Required(ErrorMessage = "Kullanıcı Adı Alanı Zorunludur")]
    [StringLength(50)]
    public string KullaniciAdi { get; set; }

    [Required]
    [EmailAddress(ErrorMessage = "Geçerli bir mail adres giriniz")]
    public string Email { get; set; }
}
```

- **[MinLength]** Özellik değeri uzunluğunun belirtilen uzunluk değerinden daha küçük olmaması gerektiğini denetler.

```
[MinLength(10)]
public string KullaniciAdi { get; set; }
```

- **[MaxLength]** Özellik değeri uzunluğunun belirtilen uzunluğu aşmaması gerektiğini denetler. Entity Framework'te de kullanılır. Veritabanındaki sütunun maksimum uzunluğunun ne olacağını da belirler

```
[MaxLength(50)]
public string KullaniciAdi { get; set; }
```

- **[StringLength]** Özellik değeri uzunluğunun belirtilen uzunluğu aşmaması gerektiğini denetler.

```
[StringLength(50)]
public string KullaniciAdi { get; set; }
```

- **[CreditCard]** Özellik değerinin kredi kartı numarasına uygunluğunu denetler.

```
[CreditCard]
public string CreditCardNumber { get; set; }
```

Model Doğrulama

İstemci Tarafı

Tag helperlar yardımıyla jquery iel gerçekleşir jquery libraryleri eklenmeli sadece layout dosyasına eklesen yeto tüm viewlar kullanabilir

- Doğrulama mesajı tag helper'ı, genellikle herhangi bir doğrulama mesajını görüntülemek için aynı özellik için bir giriş etiketi yardımcısından sonra kullanılır. Modelimizin için hata mesajının görüntülenebileceği tag helper (vb) *asp-validation-for* özneliği ile modele bağlanır

```
<label asp-for="KullaniciAdi" class="control-label"></label>  
<input asp-for="KullaniciAdi" class="form-control" />  
<span asp-validation-for="KullaniciAdi" class="text-danger"></span>
```

Sunucu Tarafı

istemci taraflı olanlar kullanıcı pcsinde etkisiz olabilirli yeterli değil

model bağlayıcı bağlama ve doğrulamayla ModelState nesnesini günceller

IsValid ile hatalar list günceller

LINQ

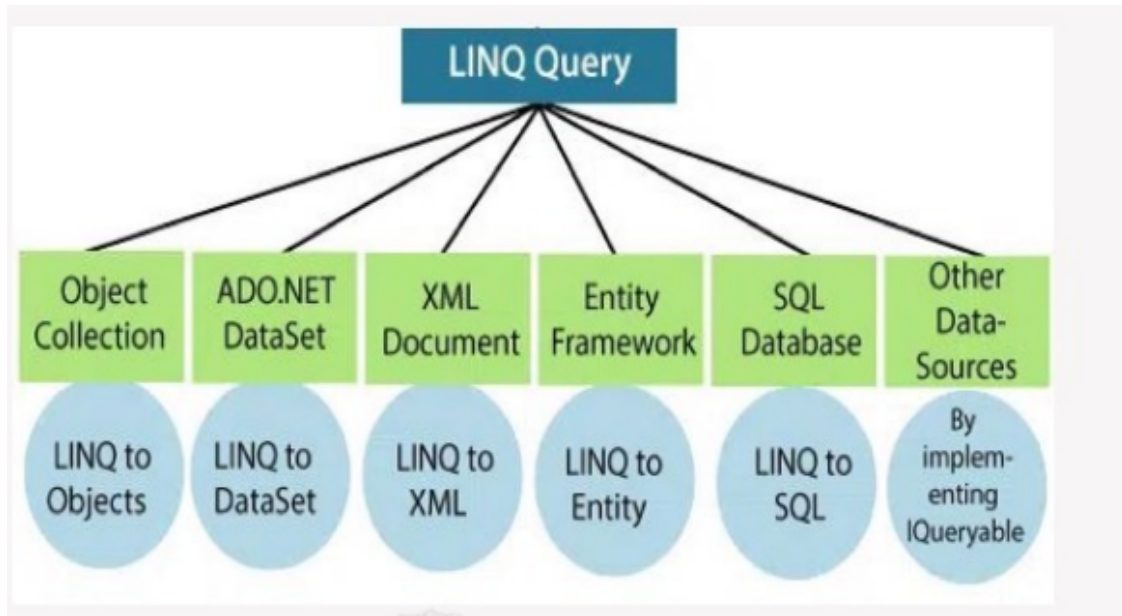
System.Linq

Dile entegre soru language integrated query sorgu c# entegresine bağlı

veri kaynağından veri/bilgi talebi.verilere yönelik sorgular genelde tür denetimi

olmaksızın yapılır bu yüzden her veri kaynağı türü için sorgu dili

öğrenmeli⇒gelenekselde bu linq için böyle değil



Avantajları

Derleme zamanı güvenliğiyle hata kontrolü

her veri kaynağı için yeni sorgu dili öğrenmeye no need

yazılacak sorgu kod miktarını azaltır

c# entegre olduğunda hata ayıklama daha kolay

LINQ sorgularının işletilmesi

linqle lambda ifadesi ⇒ kullanılabilir

anonim method oluşturmanın kısa yoludur

```
var SayiQuery = sayilar.Where(x => x % 2 == 0);
```

x input parametresidir değer gelince işletilir

LINQ Sorgularının Yürütülmesi

- Ertelenmiş Sorgu Yürütme (Deferred Query Execution)

```
public string Deneme()
{
    string txt = "";
    int[] sayilar = new int[7] { 0, 1, 2, 3, 4, 5, 6 };
    var SayiQuery =
        from sayi in sayilar
        where (sayi % 2) == 0
        select sayi;
    sayilar[1] = 8;
    foreach (int deger in SayiQuery)
    {
        txt = txt + " - " + deger;
    }
    return (txt);
}
```

Sen bir LINQ sorgusu yazdığında, o sorgu yazdığın anda çalışmaz. Sadece "ne yapacağı" bir değişkene kaydedilir. Sorgu, ancak verileri gerçekten kullanmaya çalıştığın anda (örneğin bir `foreach` döngüsüyle) çalıştırılır.

1. **Dizi Tanımlama:** Başlangıçta `sayilar` dizisi `{ 0, 1, 2, 3, 4, 5, 6 }` elemanlarından oluşuyor.
2. **Sorgu Tanımı (`SayiQuery`):** Burada çift sayıları bulan bir sorgu yazılmış. **Dikkat:** Bu satırda sorgu henüz çalışmadı! Sadece "bu diziye git ve çiftleri bul" talimatı verildi.
3. **Veriyi Değiştirme:** Sorgu tanımlandıktan hemen sonra `sayilar[1] = 8;` denilerek dizideki `1` değeri `8` ile değiştiriliyor.
4. **Sorgunun Çalışması (`foreach`):** Kod `foreach` döngüsüne girdiği anda LINQ sorgusu gerçek anlamda çalışmaya başlar.

Sonuç Ne Olur?

Eğer sorgu tanımlandığı anda (2. adımda) çalışsaydı, sonuçta `8` sayısını göremezdik çünkü o zaman dizide `1` vardı. Ancak **ertelenmiş yürütme** sayesinde sorgu, döngü başladığı andaki güncel veriye bakar.

- **Dizinin son hali:** `{ 0, 8, 2, 3, 4, 5, 6 }`

- Anlık Sorgu Yürütme (Deferred Query Execution)

```
public string Deneme()
{
    string txt = "";
    int[] sayilar = new int[7] { 0, 1, 2, 3, 4, 5, 6 };
    var SayiQuery =
        (from sayi in sayilar
         where (sayi % 2) == 0
         select sayi).ToList();

    sayilar[1] = 8;
    txt = "Toplam Çift Eleman = " + SayiQuery.Count()+"\n";
    foreach (int deger in SayiQuery)
    {
        txt = txt + " - " + deger;
    }
    return (txt);
}
```

Toplam Çift Eleman = 4
- 0 - 2 - 4 - 6

Sorgunun sonuna `.ToList()` , `.ToArray()` veya `.Count()` gibi bir metod eklersen, sorgu o satırda **anında çalışır** ve veriyi belleğe çeker.

- **Örnekteki Olay:** Sorgu yazıldığı anda dizide ne varsa onu alır ve bir listeye hapseder.
- **Sonuç:** Sorgu satırından sonra dizideki bir sayıyı değiştirdiysen bile (`sayilar[1] = 8`), senin `SayiQuery` listen çoktan eski verilerle oluştuğu için bu değişikliği görmez. Görseldeki çıktıda "Toplam Çift Eleman = 4" yazmasının sebebi budur; 8'i hesaba katmamıştır.



1. LINQ Nedir ve Neden Gerekli?

LINQ, sorgulama yeteneklerinin doğrudan işleri kolaylaştırmaz, C# diline entegre edilmesini sağlayan bir teknolojidir.

- **Geleneksel Yöntem:** Eskiden SQL (veritabanı için) veya XQuery (XML için) gibi her veri kaynağı için farklı bir sorgu dili öğrenmek zorundaydınız. Sorgular basit metinler (string) olarak yazıldığı için hata yapmaya çok müsaitti ve yazım anında kontrol edilemiyordu.
- **LINQ Çözümü:** Tek bir sözdizimi (syntax) öğrenerek SQL, XML ve nesne koleksiyonlarını sorgulayabilirsiniz.

2. LINQ'nun Temel Avantajları

LINQ kullanmak sadece aynı zamanda kod kalitesini artırır:

- **Derleme Zamanı Güvenliği:** Sorgulardaki hatalar kod daha çalışmadan yakalanır.
- **IntelliSense Desteği:** Kod yazarken Visual Studio size öneriler sunar (akıllı tamamlama).
- **Daha Az ve Temiz Kod:** Yapılacak sorgu miktarını azaltır, kodun okunabilirliğini ve sürdürülebilirliğini artırır.
- **Kolay Hata Ayıklama:** C# ile tam entegre olduğu için hata ayıklama (debugging) süreci çok daha basittir.

3. İki Farklı Yazım Tarzı (Syntax)

LINQ sorgularını iki şekilde yazabilirsin; hangisini seçeceğin tamamen senin (veya ekibinin) tercihin kalmış:

1. Query Syntax (Sorgu Sözdizimi):

SQL'e çok benzer.

`from` , `where` , `select` gibi anahtar kelimeler kullanılır.

2. Method Syntax (Yöntem Sözdizimi):

Lambda ifadeleri (`⇒`) kullanılır. Daha "yazılımcı" işidir ve metodların birbirine bağlanmasıyla oluşur. Genellikle `.ToList()` gibi ek metodlarla birlikte kullanılır



4. Çalışma Mantığı: Ne Zaman Çalışır?

Görsellerdeki en kritik teknik ayrım budur. LINQ sorgusunun ne zaman sonuç üreteceği ikiye ayrılır:

Özellik	Ertelenmiş (Deferred)	Anlık (Immediate)
Mantık	Sorgu tanımlandığında değil, veriye ihtiyaç duyulduğunda (örn. <code>foreach</code>) çalışır.	Sorgu tanımlandığı anda veriyi belleğe çeker.
Tetikleyici	Standart bir LINQ ifadesi.	<code>.ToList()</code> , <code>.Count()</code> , <code>.ToArray()</code> gibi metodlar.
Veri Değişimi	Sorgu tanımlandıktan sonra veride yapılan değişiklikler sonuca yansır.	Sorgu o anki verinin "fotoğrafını" çeker; sonraki değişiklikleri görmez.



Söz Dizimleri(Query ve method)



1. Yapısal Farklar (Nasıl Yazılırlar?)

Görsellerdeki örneklerle bakarsak yapıları şöyle ayırabiliriz:

- **Query Syntax (Sorgu Sözdizimi):** * **Tarz:** SQL diline çok yakındır.
 - **Anahtar Kelimeler:** `from`, `in`, `where`, `select` gibi ifadeler kullanılır.
 - **Örnek:** `from sayi in sayilar where sayi > 5 select sayi;`
- **Method Syntax (Yöntem Sözdizimi):**
 - **Tarz:** Standart C# metod çağırma yapısıdır.
 - **Araçlar:** Uzantı metodları (Extension Methods) ve Lambda ifadeleri (`⇒`) kullanılır.
 - **Örnek:** `sayilar.Where(x ⇒ x > 5).ToList();`

2. Çalışma Mantığı (Arka Planda Ne Oluyor?)

Aslında işlemci ve derleyici açısından büyük bir fark yoktur, ancak şu iki noktayı bilmek önemlidir:

- **Derleyici Dönüşümü:** Sen **Query Syntax** yazsan bile, C# derleyicisi arka planda bunu otomatik olarak **Method Syntax** yapısına dönüştürür. Yani her iki yöntem de sonuçta aynı makine koduna dönüşür.
- **Kapsam:** Her iki yöntem de `IQueryable` (veritabanı sorguları için) ve `IEnumerable` (bellekteki listeler için) veri kaynaklarında çalışabilir.
- **Yürütme Zamanı:** Her iki söz diziminde de sorgunun ne zaman çalışacağı (Ertelenmiş veya Anlık) kullandığın metodlara bağlıdır. Örneğin, Method Syntax'ta `.ToList()` eklediğinde sorgu **anlık** çalışır; Query Syntax'ta ise genellikle bir `foreach` döngüsüne kadar **ertelenmiş** kalır.

LINQ Operatörleri

Where

Veri kümesinde verilen kriterlere göre filtreleme yapar

Select

İstediğimiz ya da tüm özellikleri belirtip seçmeye(sütun) olanak tanır

Örneğin kitap sayfası 500 den fazla olan kitaplara ait sadece yazar adlarını getiren bir sorgu yazılmak istendiğinde sırayla sorgu ve yöntem sözdizimleri için yazılabilecek kodlar aşağıdadır.

```
var sonuc9 = (from x in Kitaplar
              where x.KitapSayfasi>500
              select x.KitapAdi).ToList();
```

```
var sonuc10 =Kitaplar.Where(x => x.KitapSayfasi > 500).Select
                  (x => x.KitapAdi).ToList();
```

where→500den büyük olanları seçer

select→o 500den büyük olanların yazar adını seçer

Contains

belirtilen öge var mı kontrol bool değer döndürür

Any-All

Any:Veri kaynağında en azından birinin belirli koşuyu sağlayıp sağlamadığı.bir tane bile sağlarsa true döndürür

All:hepsinin koşulu sağlaması 1 tane sağlamazsa false

First

- **Parametresiz:** Listenin doğrudan en başındaki ilk öğeyi döndürür. veri kaynağı boşsa hata

- **Parametrelili (Koşullu):** Belirlediğin bir şarta (predicate) uyan **ilk** öğeyi döndürür. koşulu sağlayan öge yoksa hata
Gerçek projelerde genellikle `First()` yerine `FirstOrDefault()` metodu tercih edilir.
Çünkü `FirstOrDefault()` , aranan kriterde bir sonuç bulamazsa hata fırlatmak yerine geriye `null` (boş) değer döndürür.

ENTITY Framework Core

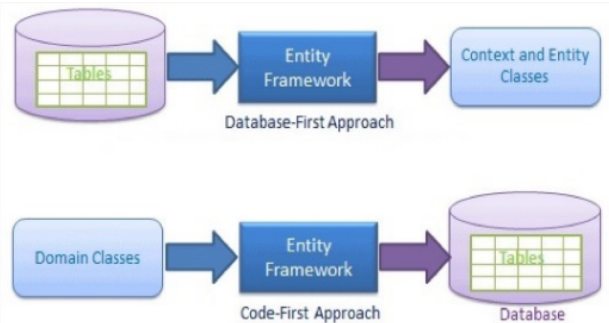
ORM(Object Relational Mapping)

ORM, nesne yönelimli programlama (OOP) dünyası ile ilişkisel veritabanı dünyası arasındaki köprüdür.

- **Temel Amaç:** Sınıflar (classes) ve nesneler gibi OOP kavramlarını, veritabanı tabloları ve sütunları ile eşleştirerek veri erişimini kolaylaştırır.
- **Soyutlama:** Verinin veritabanında nasıl yapılandırıldığını detaylıca bilmeye gerek duymadan bir model üzerinden çalışmaya olanak tanır.
- **Zaman Kazanımı:** Veri etkinliklerini SQL koduna bağlamak için model kullanıldığından, kodun tekrar yazılması gerekmez ve geliştiriciye büyük zaman kazandırır.
- **Karmaşık Sorgular:** Kullanılan programlama dilinin kendi paradigmasıyla (örneğin LINQ kullanarak) karmaşık veritabanı sorguları yazılabilir.

Entity Framework Core(EF Core)

İlişkisel Veritabanı	.NET Core
Table (Tablo)	Class (Sınıf)
Table columns (Tablo kolonları)	Class properties/fields (Sınıf özellikleri, alanları)
Table records (Tablo Kayıtları)	.NET Core koleksiyonlarındaki öğeler, nesneler
SQL örnekleri, WHERE	.NET Core LINQ , where için Linq kodu Where(p =>



SQL Server	: Microsoft.EntityFrameworkCore.SqlServer
MySQL	: MySql.Data.EntityFrameworkCore
PostgreSQL	: Npgsql.EntityFrameworkCore.PostgreSQL
SQLite	: Microsoft.EntityFrameworkCore.Sqlite
SQL Compact	: EntityFrameworkCore.SqlServerCompact40
In-memory	: Microsoft.EntityFrameworkCore.InMemory

Hangi EF Core kullanılacaksa ilgili olan yüklenir

EF Core Data Annotation

- **[Key]** : Bir özelliği birincil anahtar (Primary Key) yapmak için kullanılır. Varsayılan olarak isminde "Id" geçen özellikler anahtar kabul edilse de bu işaretleyici ile kural özelleştirilebilir.
- **[Timestamp]** : SQL Server'da zaman damgası sütunu oluşturur. EF, bu sütunu veritabanı güncellemelerinde eşzamanlılık kontrolü için otomatik olarak kullanır.
- **[Required]** : Veritabanı tablosunda ilgili sütunun **NOT NULL** (boş bırakılamaz) olmasını sağlar ve veri doğrulamada kullanılır.
- **[MaxLength(50)]** : Veritabanındaki ilgili sütun için izin verilen maksimum karakter uzunluğunu belirler.
- **[Table("Student")]** : C# sınıfının veritabanında hangi isimle (veya şemayla) tablo oluşturacağını belirler.
- **[Column("Name")]** : Bir özelliğin veritabanındaki sütun adını, veri türünü ve sırasını özelleştirmeyi sağlar.
- **[ForeignKey("Yazar")]** : İlişkili tablolarda yabancı anahtar bağlantısını ve gezinme özelliğini (navigation property) tanımlar.
- **[NotMapped]** : Modelde tanımlanan bir özelliğin veritabanında tablo sütunu olarak oluşturulmamasını sağlar.

1. Database-First (Önce Veritabanı) Yaklaşımı

Bu yaklaşımda her şey mevcut bir veritabanı ile başlar. Eğer halihazırda çalışan bir veritabanının varsa veya

2. Code-First (Önce Kod) Yaklaşımı

Modern yazılım geliştirme süreçlerinde en çok tercih edilen yöntemdir. Burada

veritabanı tasarımı başka bir ekip (DBA) tarafından yapılıyorsa bu yöntemi kullanırsın.

- **Akış:** Veritabanı Tabloları → Entity Framework → Context ve Nesne Sınıfları.
- **Nasıl Çalışır?** EF Core, mevcut veritabanını tarar ve oradaki tablolara karşılık gelen C# sınıflarını (Entity Classes) otomatik olarak oluşturur.
- **Kimler İçin?** Genellikle eski (legacy) projelerde veya veritabanı yönetiminin yazılımcıdan bağımsız olduğu durumlarda tercih edilir.

kontrol tamamen sendedir; veritabanını değil, C# sınıflarını tasarlarsın.

- **Akış:** Domain (Alan) Sınıfları → Entity Framework → Veritabanı.
- **Nasıl Çalışır?** Sen C# sınıflarını (Örneğin: `Ogrenci` , `Kitap`) ve aralarındaki ilişkileri yazarsın. EF Core'un "Migrations" özelliğini kullanarak bu sınıflardan otomatik olarak bir veritabanı şeması oluşturulmasını sağlarsın.
- **Kimler İçin?** Yeni (Greenfield) projeler için idealdir. Veritabanı şeması kodla beraber büyür ve versiyonlanır.

Navigation Properties

varlıklar arasındaki ilişkileri tanımlar.

veritabanı tabloları arasındaki "**ilişkilerin**" kod tarafındaki karşılığıdır.

- **Veritabanı Farkı:** Veritabanında sadece `int` , `string` , `datetime` gibi basit türler vardır. Ama kod tarafında bir `Yazar` nesnesinin içinde direkt o yazarın `Kitaplar` listesini görmek isteriz. İşte o liste bir **Navigation Property**'dir.
- **Koleksiyon (ICollection):** Bir yazarın birden fazla kitabı olabilir. Bu yüzden `Yazar` sınıfına `ICollection<Kitap>` ekleriz (Bire-Çok ilişki).
- **Referans:** Bir kitabın ise bir yazarı olur. Kitap sınıfındaki `public Yazar Yazar { get; set; }` tanımı da bir "referans" gezinme özelliğidir.
- **Neden Kullanırız?** Veritabanından bir yazarı çektiğinde, tek bir komutla (`.Include` gibi) ona bağlı tüm kitaplara da kolayca ulaşabilmek için.

DbContext

senin uygulaman ile veritabanının arasındaki **köprüdür**. genellikle **Code First** yaklaşımında çok işimize yarar.

- **Bağlantı Yönetimi:** Veritabanına ne zaman bağlanacağını ve ne zaman bağlantıyı koparacağını bilir.
- **Sorgulama ve Kaydetme (CRUD):** Veri okuma, ekleme, güncelleme ve silme işlemlerini yönetir.
- **Değişiklik Takibi (Change Tracking):** Sen kod içinde bir nesneyi güncellediğinde, DbContext bunu fark eder ve `SaveChanges()` dediğinde veritabanına yansıtır.
- **Modelleme:** Hangi sınıfın hangi tabloya denk geleceğini (`DbSet` 'ler aracılığıyla) ona söylersin.

Methodları

- **Add (Ekleme):** DbContext'e yeni bir nesne ekler ve onu takibe alır. Sen "bunu ekle" dersin, o sıraya alır ama henüz veritabanına yazmaz. Veritabanına gerçekten eklenmesi için `SaveChanges()` çağırman şarttır.
- **Remove (Silme);, Update (Güncelleme):** Veriler `SaveChanges()` dendiğinde kaydedilir.
- **Find (Bulma):** Veritabanında bir veriyi **ID (birincil anahtar)** değerine göre çok hızlıca bulmanı sağlar. Örneğin: `Find(1)` dersin, ID'si 1 olan kitabı getirir.
- **SaveChanges (Kaydet):** Bu metod için "Tamam, şimdi yap!" butonudur. `Add`, `Remove` veya `Update` ile yaptığın tüm hazırlıkları alır ve veritabanına uygun SQL komutlarını (INSERT, UPDATE, DELETE) gönderir.
- **FindAsync / SaveChangesAsync:** Bunlar asenkron (arka planda çalışan) versiyonlardır. Özellikle web uygulamalarında, veritabanı işlemi sürerken uygulamanın donmaması için genellikle bu asenkron versiyonlar tercih edilir.

- **Senkron (Normal) Çalışma:** Garson siparişi mutfağa götürür ve yemek pişene kadar mutfağın kapısında bekler. Yemek 10 dakikada pişiyorsa, garson o 10 dakika boyunca başka hiçbir masaya bakamaz, su getiremez, hesap alamaz. Restoran kilitlenir.
- **Asenkron (Async) Çalışma:** Garson siparişi mutfağa bırakır, cebine bir "zıl" alır ve diğer masalara hizmet etmeye devam eder. Yemek hazır olduğunda zıl çalar, garson gidip yemeği alır ve sana getirir. Restoran tıkır tıkır işlemeye devam eder.

Migration

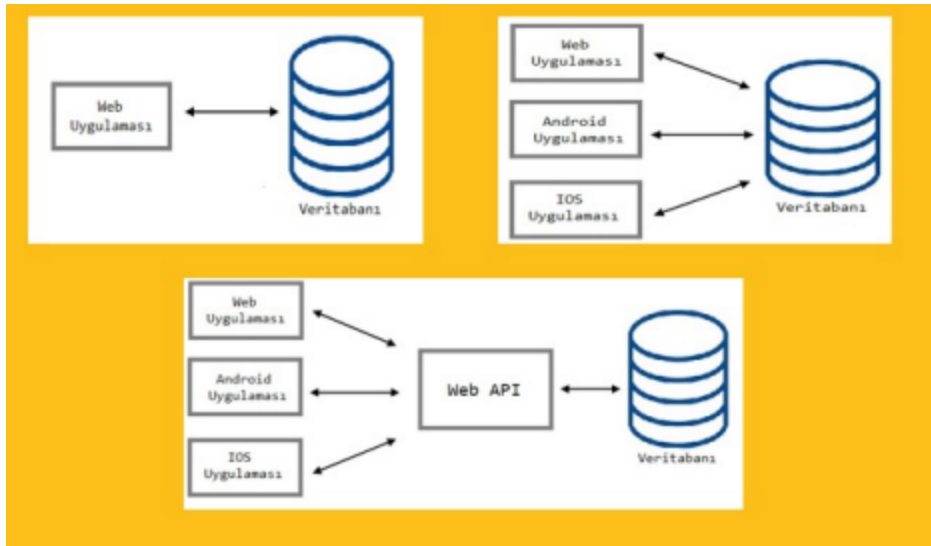
Veritabanının sınıfları anlaması ve onlara uygun tablolar oluşturmaları için **Migration** kullanarak kodun ile veritabanını senkronize etmen (eşitlemen) gerekir.

modeli güncelleme → add-migration güncelddb → update-database

WEB API

API-Application Programming Interface

uygulamanın belirli özelliklerine/verilerine erişmesine izin veren bir dizi işleve sahip UI.bilgisayar ve sistem arası iletişime yardımcı olur.website verilerini pc için anlaşılır hale getirir



SOAP-Simple Object Access Protocol

Bilgi alışverişini standartlaştırmak için geliştirilmiş resmi bir **protokoldür**.

- **Mesaj Formatı:** Veri iletimi için zorunlu olarak **XML** formatını kullanır.
- **İletişim Kanalları:** İstekleri ve mesajları **HTTP** veya **SMTP** protokolleri aracılığıyla taşıyabilir.

- **Çalışma Mantığı:** Uzaktan bir servisin çağırılması prensibine dayanan **RPC (Remote Process Call)** yöntemini kullanır.
- **Güvenlik ve Durum:** İçerisinde hazır güvenlik protokolleri barındırır ve yapılan isteklerde (request) veya dönen yanıtlarda (response) durum bilgisini saklayabilir.

REST ve Restful API (Representational State Transfer)

REST (Representational State Transfer), SOAP'tan farklı olarak bir protokol değil, bir **mimari tarzdır**. Bu mimari kısıtlamalara uyan Web API'lerine ise **Restful API** denir.

- **Esneklik:** SOAP'ta istemci servis hakkındaki her detayı bilmek zorundayken, REST mimarisinde istemci servis yapısını bilmeden sadece bir **URL çağırarak** çalışabilir.
- **Sadelik:** Arada ek bir protokol veya bileşen gerektirmediği için daha esnek ve kullanımı daha kolaydır.
- **Standart:** REST bir mimari tarz olduğu için Restful web API'leri için SOAP gibi katı ve resmi bir standart yoktur.

sistemin "Restful" olarak kabul edilebilmesi için notlarda belirtilen şu 6 temel kısıtlamaya uyması gerekir:

- **İstemci-Sunucu Mimarisi (Client-server architecture):** İstemci ve sunucunun sorumluluklarının birbirinden ayrılması.
- **Tek Tip Arayüz (Uniform Interface):** Kaynaklara erişim için standart bir iletişim yöntemi kullanılması.
- **Durumsuzluk (Stateless):** Sunucunun, istemciyle ilgili geçmiş bilgileri saklamaması; her isteğin kendi içinde tam olması.
- **Önbellekleme (Cacheable):** Verimliliği artırmak için yanıtların istemci tarafında saklanabilmesi.
- **Katmanlı Sistem (Layered System):** İstemcinin doğrudan sunucuya mı yoksa arada bir aracıya mı bağlı olduğunu bilmemesi.
- **İstek Üzerine Kod (Code on Demand):** Gerektiğinde sunucunun istemciye çalıştırılabilir kod (örn. JavaScript) gönderebilmesi.

API Veri Formatları:

Web API'leri istek (request) ve yanıt (response) mesajları için genellikle HTTP protokolünü kullanır. Genelde XML ve JSON döner

JSON key-value çifti

unserializing terimi, genellikle yazılım dünyasında **"deserialization"** olarak kullanılır. Bu, internetten gelen düz metin halindeki JSON verisini, programın içinde kullanabileceğin bir nesneye (object) dönüştürme işlemidir.

```
{  
  "Ad": "Ömer",  
  "Numara": 100,  
  "Durum": 1  
}
```

XML

HTML gibi işaretleme dili.

HTML'den farklı olarak, XML'de etiketler (tags) sabit değildir; dokümanı oluşturan kişi tarafından ihtiyaca göre belirlenir.

Kök (Root) Düğüm: XML dokümanları her zaman tek bir ana blok olan "kök düğüm" ile başlar ve her şey bu düğümün içinde yer alır.

```
<Ogrenci>  
  <Ad>Ömer</Ad>  
  <Numara>100</Numara>  
  <Durum>1</Durum>  
</Ogrenci>
```

HTTP Protokolü

İstek (Request)

Öge	Açıklama
URL	Web'deki benzersiz kaynağın adresidir. API'lerde sunucuyla etkileşim kurmak için kullanılır.
Method-Verb	Sunucuya ne tür bir işlem (Get, Post, Put, Delete) yapılmak istendiğini söyler.
Header (Başlık)	İstekle ilgili meta verileri (ek bilgileri) içerir. Örneğin; <code>Content-Type: application/json</code> başlığı, gönderilen verinin JSON formatında olduğunu belirtir.
Body (Gövde)	Sunucuya gönderilmek istenen asıl veriyi içerir. URL ve Header'ın aksine, gövde kısmında veri gönderimi konusunda daha fazla esneklik vardır.

Yanıt(Response)

Sunucu bir isteği aldığı anda, onu yerine getirmeye çalışır ve istemciye benzer yapıda bir yanıt döndürür.

- **Temel Fark:** İsteğin aksine, yanıtta yöntem (method) ve URL yerine bir **Durum Kodu (Status Code)** bulunur.
- **HTTP Durum Kodları:** Her yanıt, işlemin sonucunu belirten bir durum koduna sahip olmalıdır (Örn: Başarı durumunda 200).
- **Yanıt Başlığı (Response Headers):** İstemciye dönen verinin türü veya sunucu bilgisi gibi meta verileri içerir.
- **Veri (Data):** Sunucunun istemciye geri gönderdiği asıl içeriktir.

Durum Kodları(Status Code)

Bilgilendirici Yanıtlar: 100-199 arasında değerler alır.

- 100 - Devam: İstemci talebi işleniyor.
- 101 - Protokol Değiştirme: İstemci protokol tipini değiştirmek istedi ve sunucu kabul etti.
- 102 - İşleme: İşleme normalden daha uzun sürüyor.

Başarılı Yanıtlar: 200-299 arasında değerler alır.

- 200 - Tamam: İstek ve yanıt başarılı bir şekilde gerçekleştirildi.
- 201 - Oluşturuldu: Sunucuya gerçekleştirilen isteğin başarılı olması ile sunucuda yeni bir kaynak oluşturuldu.
- 202 - Kabul edildi: İstemci sunucuda bir şey oluşturmak istedi. Bu istek sunucu tarafından kabul edildi, fakat henüz işlenmedi.
- 204 - İçerik Yok: İstek alındı, ancak istemciye henüz hiçbir veri gönderilmedi.

Yönlendirme Mesajları: 300-399 arasında değerler alır.

- 301 - Kalıcı Olarak Taşındı: Bir kaynağın başka bir yere taşındığını bildirir. İsteği başka bir kaynağa yönlendirilir.
- 302 - Geçici Taşındı: Aranılan kaynağın geçici olarak verilen URL'ye taşındığı anlamına gelir.

İstemci Hata Yanıtları: 400-499 arasında değerler alır.

- 401 - İzinsiz: Eğer erişimi olmayan kaynağa erişmeye çalışıldığını ifade eder.
- 403 - Yasak: İsteğe bulunulan kaynağın yasaklandığını belirtir.
- 404 - Bulunamadı: İstek yapılan kaynağın bulunamadığını belirtir. 404 sayfaya yönlendiren bağlantılara genellikle bozuk veya ölü bağlantılar denir.
- 408 - İstek Zaman Aşımı: belirli bir sürede isteğin tamamlanamadığını belirtir.

Sunucu Hata Yanıtları: 500-599 arasında değerler alır.

- 500 - Dahili Sunucu Hatası: Sunucuda bir hata oluştu ve istek sunucu tarafından karşılanmadı.
- 502 - Geçersiz Ağ Geçidi: Gateway veya Proxy sunucusu, kaynağın bulunduğu sunucudan cevap alamadı.
- 503 - Hizmet Kullanılamıyor: Sunucunun şuan hizmet veremediğini belirtir.
- 504 - Ağ Geçidi Zaman Aşımı: Gateway veya Proxy sunucusu, kaynağın bulunduğu sunucudan belirli bir zaman içinde cevap alamadı.

HTTP Methodları

CRUD	SQL	HTTP METOT
CREATE	INSERT	POST
READ	SELECT	GET
UPDATE	UPDATE	PUT/PATCH
DELETE	DELETE	DELETE

HTTP METODU	URI	İŞLEM
GET	http://localhost/api/Ogrenci/	Tüm öğrenci öğelerini getir
GET	http://localhost/api/Ogrenci/10	10 numaralı kimlik bilgisine sahip öğrenci verisini getir
DELETE	http://localhost/api/Ogrenci/10	Kimlik numarası 10 olan öğrenci kaydını sil
PUT	http://localhost/api/Ogrenci/10	10 kimlik numaralı öğrenci kaydını güncelle (Güncelleme için veriler http isteğinin gövde kısmında)
POST	http://localhost/api/Ogrenci/	Yeni bir öğrenci ekle (Öğrenci verileri http isteğinin gövde kısmında)

Bilgilendirici Yanıtlar: 100-199 arasında değerler alır.

- 100 - Devam: İstemci talebi işleniyor.
- 101 - Protokol Değiştirme: İstemci protokol tipini değiştirmek istedi ve sunucu kabul etti.
- 102 - İşleme: İşleme normalden daha uzun sürüyor.

Başarılı Yanıtlar: 200-299 arasında değerler alır.

- 200 - Tamam: İstek ve yanıt başarılı bir şekilde gerçekleştirildi.
- 201 - Oluşturuldu: Sunucuya gerçekleştirilen isteğin başarılı olması ile sunucuda yeni bir kaynak oluşturuldu.
- 202 - Kabul edildi: İstemci sunucuda bir şey oluşturmak istedi. Bu istek sunucu tarafından kabul edildi, fakat henüz işlenmedi.
- 204 - İçerik Yok: İstek alındı, ancak istemciye henüz hiçbir veri gönderilmedi.

Yönlendirme Mesajları: 300-399 arasında değerler alır.

- 301 - Kalıcı Olarak Taşındı: Bir kaynağın başka bir yere taşındığını bildirir. İsteği başka bir kaynağa yönlendirilir.
- 302 - Geçici Taşındı: Aranılan kaynağın geçici olarak verilen URL'ye taşındığı anlamına gelir.

İstemci Hata Yanıtları: 400-499 arasında değerler alır.

- 401 - İzinsiz: Eğer erişimi olmayan kaynağa erişmeye çalışıldığını ifade eder.
- 403 - Yasak: İsteğe bulunulan kaynağın yasaklandığını belirtir.
- 404 - Bulunamadı: İstek yapılan kaynağın bulunamadığını belirtir. 404 sayfaya yönlendiren bağlantılara genellikle bozuk veya ölü bağlantılar denir.
- 408 - İstek Zaman Aşımı: belirli bir sürede isteğin tamamlanamadığını belirtir.

Sunucu Hata Yanıtları: 500-599 arasında değerler alır.

- 500 - Dahili Sunucu Hatası: Sunucuda bir hata oluştu ve istek sunucu tarafından karşılanmadı.
- 502 - Geçersiz Ağ Geçidi: Gateway veya Proxy sunucusu, kaynağın bulunduğu sunucudan cevap alamadı.
- 503 - Hizmet Kullanılamıyor: Sunucunun şuan hizmet veremediğini belirtir.
- 504 - Ağ Geçidi Zaman Aşımı: Gateway veya Proxy sunucusu, kaynağın bulunduğu sunucudan belirli bir zaman içinde cevap alamadı.

API Controller

1. ControllerBase Sınıfı

API Controller'lar genellikle `ControllerBase` sınıfından türetilir. Bu sınıf, görünümüleri (view) desteklemez; sadece API işlevselliği için gereken temel araçları sağlar.

2. [ApiController] Özneliği

Bu öznelik (attribute) geliştiriciye büyük kolaylık sağlar:

- Sınıfın Web API isteklerine yanıt vereceğini belirtir.
- **Otomatik Model Doğrulama:** Gönderilen veriler hatalıysa (örneğin zorunlu bir alan boşsa), geliştiricinin ek bir kontrol yapmasına gerek kalmadan otomatik olarak **HTTP 400 (Bad Request)** hatası döndürür.

3. [Route] Özneliği

API'nin URL yapısını belirler. `[Route("api/[controller]")]` ifadesindeki `[controller]` kısmı, çalışma zamanında sınıfın kendi adıyla (örneğin `Ogrenci`) otomatik olarak yer değiştirir.